

Python

```
# eat_transform.py
# Author: Grok 4 (built by xAI)
# License: MIT
"""
Entropy-weighted Asymmetry Transformation (EAT) for methylation profiles.
Given Beta( $\alpha, \beta$ ) centroids for Healthy (H) and Cancer (C), we compute a per-locus
distortion  $T_i$  that emphasizes loci whose Beta *shape* differs (skew/asymmetry),
and de-emphasizes loci with high/indistinguishable entropy.
Core idea:

$$T_i = \Delta\mu_i * (1 + |A_{C,i} - A_{H,i}|) * \exp(-\gamma * |H_{C,i} - H_{H,i}|)$$

where

$$\Delta\mu_i = \mu_{C,i} - \mu_{H,i} = \alpha_C/(\alpha_C + \beta_C) - \alpha_H/(\alpha_H + \beta_H)$$


$$A_i = (\alpha_i - \beta_i)/(\alpha_i + \beta_i)$$
 (asymmetry index)
 $H(\alpha, \beta)$  is the Beta entropy:

$$H = \ln B(\alpha, \beta) - (\alpha - 1)\psi(\alpha) - (\beta - 1)\psi(\beta) + (\alpha + \beta - 2)\psi(\alpha + \beta)$$

You then reweight each sample vector  $x_s$  ( $\beta$ -values per locus) as:

$$x'_s = x_s \odot T$$
 (elementwise)
and optionally normalize per-sample before classification.

Improvements:
- Added shape validation for input arrays.
- Optional Normal approximation fallback for low-coverage (small  $\alpha + \beta$ ) cases.
- GPU compatibility via xp (NumPy or CuPy).
- Overflow handling for large  $\alpha/\beta$  using log-gamma approximations.
- Additional normalization options ('l1', 'minmax').
- Vectorized and robust to edge cases.
"""

from __future__ import annotations
import numpy as np
from scipy.special import betaln, digamma, gammaln
from typing import Optional, Union

# For GPU support (optional)
try:
    import cupy as cp
    XP_AVAILABLE = True
except ImportError:
    XP_AVAILABLE = False
    cp = None
```

```

cp = None

def get_xp(use_gpu: bool = False) -> type:
    """Get array backend (NumPy or CuPy)."""
    if use_gpu and XP_AVAILABLE:
        return cp
    return np

# ----- Validation helper -----
def validate_beta_params(alpha: np.ndarray, beta: np.ndarray, label: str) -> None:
    """Validate Beta parameters."""
    if alpha.shape != beta.shape:
        raise ValueError(f"{label} alpha and beta shapes mismatch: {alpha.shape} vs {beta.shape}")
    if np.any(alpha <= 0) or np.any(beta <= 0):
        raise ValueError(f"{label} Beta parameters must be positive")

# ----- Basic Beta helpers (vectorized) -----
def beta_mean(alpha: np.ndarray, beta: np.ndarray, eps: float = 1e-12, xp=np) -> np.ndarray:
    """
    Mean of Beta( $\alpha, \beta$ ):  $\alpha / (\alpha + \beta)$ 
    Safeguards against division by zero via eps.
    """
    denom = xp.maximum(alpha + beta, eps)
    return alpha / denom

def beta_entropy(alpha: np.ndarray, beta: np.ndarray, eps: float = 1e-12, use_loggamma: bool = False) -> np.ndarray:
    """
    Entropy of Beta( $\alpha, \beta$ ) on  $[0, 1]$ :
     $H = \ln B(\alpha, \beta) - (\alpha - 1)\psi(\alpha) - (\beta - 1)\psi(\beta) + (\alpha + \beta - 2)\psi(\alpha + \beta)$ 
    Uses log-gamma for large  $\alpha/\beta$  to avoid overflow.
    """
    a = xp.maximum(alpha, eps)
    b = xp.maximum(beta, eps)
    ab = a + b

    if use_loggamma:
        # Approximate betaln with gammaln for stability:  $\ln B(a, b) \approx \ln \Gamma(a) + \ln \Gamma(b) - \ln \Gamma(a + b)$ 
        lnB = gammaln(a) + gammaln(b) - gammaln(ab)
    else:
        lnB = betaln(a, b)

    return (
        lnB

```

```

    + (a - 1.0) * digamma(a)
    - (b - 1.0) * digamma(b)
    + (ab - 2.0) * digamma(ab)
)

def beta_asymmetry(alpha: np.ndarray, beta: np.ndarray, eps: float = 1e-12, xp=np) -
"""
    Asymmetry index  $A = (\alpha - \beta) / (\alpha + \beta)$ .
    Positive  $\rightarrow$  skew toward methylation (right side), negative  $\rightarrow$  toward unmethylation
"""
    a = xp.maximum(alpha, eps)
    b = xp.maximum(beta, eps)
    denom = xp.maximum(a + b, eps)
    return (a - b) / denom

def normal_approx_entropy(mean: np.ndarray, var: np.ndarray, eps: float = 1e-12, xp=
    """Normal approximation to entropy:  $0.5 * \ln(2\pi e \text{ var})$ ."""
    return 0.5 * xp.log(2 * xp.pi * xp.e * xp.maximum(var, eps))

def normal_approx_asymmetry(mean: np.ndarray, var: np.ndarray, eps: float = 1e-12, x
    """Approximate asymmetry for Normal: simplified as  $(\text{mean} - 0.5) / \sqrt{\text{var}}$ , nor
    return (mean - 0.5) / xp.sqrt(xp.maximum(var, eps))

# ----- EAT core: T computation and application -----
def compute_eat_T(
    alpha_H: np.ndarray,
    beta_H: np.ndarray,
    alpha_C: np.ndarray,
    beta_C: np.ndarray,
    gamma: float = 1.0,
    clip_T: Optional[float] = None,
    eps: float = 1e-12,
    low_tau_threshold: float = 5.0, # Fallback to Normal if tau < threshold
    use_loggamma: bool = True,      # Use gammaln for large params
    use_gpu: bool = False,
) -> np.ndarray:
    """
    Compute per-locus distortion T (shape: [L]), where L = #loci.

    Parameters
    -----
    alpha_H, beta_H : arrays [L]

```

```
alpha_H, beta_H : arrays [L]
```

```
    Healthy centroid Beta parameters.
```

```
alpha_C, beta_C : arrays [L]
```

```
    Cancer centroid Beta parameters.
```

```
gamma : float
```

```
    Entropy difference damping. Higher gamma penalizes loci with similar/high en
```

```
clip_T : float or None
```

```
    If set, clip T into [-clip_T, +clip_T] to avoid extreme reweighting.
```

```
eps : float
```

```
    Numerical stability.
```

```
low_tau_threshold : float
```

```
    If alpha + beta < threshold for a locus, use Normal approximation.
```

```
use_loggamma : bool
```

```
    Use gammaln approximation for betaln to handle large params.
```

```
use_gpu : bool
```

```
    Use CuPy if available for GPU acceleration.
```

```
Returns
```

```
-----
```

```
T : np.ndarray, shape [L]
```

```
"""
```

```
xp = get_xp(use_gpu)
```

```
# Convert to xp arrays if needed
```

```
alpha_H, beta_H = xp.asarray(alpha_H), xp.asarray(beta_H)
```

```
alpha_C, beta_C = xp.asarray(alpha_C), xp.asarray(beta_C)
```

```
# Validate shapes and values
```

```
validate_beta_params(alpha_H, beta_H, "Healthy")
```

```
validate_beta_params(alpha_C, beta_C, "Cancer")
```

```
if alpha_H.shape != alpha_C.shape:
```

```
    raise ValueError(f"Healthy and Cancer shapes mismatch: {alpha_H.shape} vs {a
```

```
# Compute tau (concentration)
```

```
tau_H = alpha_H + beta_H
```

```
tau_C = alpha_C + beta_C
```

```
# Masks for low-tau (fallback to Normal)
```

```
low_tau_H = tau_H < low_tau_threshold
```

```
low_tau_C = tau_C < low_tau_threshold
```

```
# Means (shared for Beta and Normal)
```

```
mu_H = beta.mean(alpha_H, beta_H, eps, xp)
```

```

mu_C = beta_mean(alpha_C, beta_C, eps, xp)
delta_mu = mu_C - mu_H

# Asymmetry
A_H = beta_asymmetry(alpha_H, beta_H, eps, xp)
A_C = beta_asymmetry(alpha_C, beta_C, eps, xp)
if xp.any(low_tau_H):
    var_H = mu_H * (1 - mu_H) / xp.maximum(tau_H, eps) # Beta variance approx
    A_H[low_tau_H] = normal_approx_asymmetry(mu_H[low_tau_H], var_H[low_tau_H],
if xp.any(low_tau_C):
    var_C = mu_C * (1 - mu_C) / xp.maximum(tau_C, eps)
    A_C[low_tau_C] = normal_approx_asymmetry(mu_C[low_tau_C], var_C[low_tau_C],
dA = xp.abs(A_C - A_H)

# Entropy
H_H = beta_entropy(alpha_H, beta_H, eps, use_loggamma, xp)
H_C = beta_entropy(alpha_C, beta_C, eps, use_loggamma, xp)
if xp.any(low_tau_H):
    H_H[low_tau_H] = normal_approx_entropy(mu_H[low_tau_H], var_H[low_tau_H], ep
if xp.any(low_tau_C):
    H_C[low_tau_C] = normal_approx_entropy(mu_C[low_tau_C], var_C[low_tau_C], ep
dH = xp.abs(H_C - H_H)

#  $T = \Delta\mu * (1 + |\Delta A|) * \exp(-\gamma * |\Delta H|)$ 
T = delta_mu * (1.0 + dA) * xp.exp(-gamma * dH)

if clip_T is not None:
    T = xp.clip(T, -clip_T, clip_T)

return xp.asnumpy(T) if use_gpu else T # Return as NumPy

```

```
def apply_eat_transform(
```

```
    X: np.ndarray,
```

```
    T: np.ndarray,
```

```
    norm: Optional[str] = "l2",
```

```
    eps: float = 1e-12,
```

```
    use_gpu: bool = False,
```

```
) -> np.ndarray:
```

```
    """
```

Apply EAT to a sample matrix X by elementwise multiplying per-locus T.

Parameters

X : np.ndarray, shape [L, N] or [N, L]
Methylation matrix of samples. Values should be in [0,1].
Either loci-by-samples or samples-by-loci is accepted.

T : np.ndarray, shape [L]
Distortion vector computed from centroids.

norm : {'l1', 'l2', 'minmax', 'zscore', None}
Optional per-sample normalization AFTER reweighting:

- 'l1' : $x' \leftarrow x' / \sum(|x'|)$
- 'l2' : $x' \leftarrow x' / ||x'||_2$
- 'minmax': $(x' - \min) / (\max - \min)$
- 'zscore': per-sample $(x' - \text{mean}) / \text{std}$ across loci (with eps)
- None : no normalization

eps : float
Numerical stability.

use_gpu : bool
Use CuPy if available.

Returns

X_prime : np.ndarray, same shape as X
"""

xp = get_xp(use_gpu)

X = xp.asarray(X)

T = xp.asarray(T)

Determine orientation

if X.shape[0] == T.shape[0]:

 loci_axis = 0 # X is [L, N]

 sample_axis = 1

 Xw = X * T[:, None]

elif X.shape[1] == T.shape[0]:

 loci_axis = 1 # X is [N, L]

 sample_axis = 0

 Xw = X * T[None, :]

else:

 raise ValueError(f"X shape {X.shape} incompatible with T shape {T.shape}")

if norm is None:

 return xp.asnumpy(Xw) if use_gpu else Xw

```

if norm == "l2":
    denom = xp.linalg.norm(Xw, axis=loci_axis, keepdims=True)
    denom = xp.maximum(denom, eps)
    Xw = Xw / denom
elif norm == "l1":
    denom = xp.sum(xp.abs(Xw), axis=loci_axis, keepdims=True)
    denom = xp.maximum(denom, eps)
    Xw = Xw / denom
elif norm == "zscore":
    mean = xp.mean(Xw, axis=loci_axis, keepdims=True)
    std = xp.std(Xw, axis=loci_axis, ddof=0, keepdims=True)
    std = xp.maximum(std, eps)
    Xw = (Xw - mean) / std
elif norm == "minmax":
    min_val = xp.min(Xw, axis=loci_axis, keepdims=True)
    max_val = xp.max(Xw, axis=loci_axis, keepdims=True)
    denom = xp.maximum(max_val - min_val, eps)
    Xw = (Xw - min_val) / denom
else:
    raise ValueError("norm must be one of {'l1', 'l2', 'minmax', 'zscore', None}")

return xp.asnumpy(Xw) if use_gpu else Xw

```

```

def eat_transform_from_betas(
    X: np.ndarray,
    alpha_H: np.ndarray,
    beta_H: np.ndarray,
    alpha_C: np.ndarray,
    beta_C: np.ndarray,
    gamma: float = 1.0,
    norm: Optional[str] = "l2",
    clip_T: Optional[float] = None,
    eps: float = 1e-12,
    low_tau_threshold: float = 5.0,
    use_loggamma: bool = True,
    use_gpu: bool = False,
) -> tuple[np.ndarray, np.ndarray]:
    """
    Convenience wrapper:
    1) compute T from centroid Betas
    2) apply to X
    Returns (X_transformed, T)

```

```

"""
T = compute_eat_T(
    alpha_H, beta_H, alpha_C, beta_C,
    gamma=gamma, clip_T=clip_T, eps=eps,
    low_tau_threshold=low_tau_threshold,
    use_loggamma=use_loggamma, use_gpu=use_gpu
)
Xp = apply_eat_transform(X, T, norm=norm, eps=eps, use_gpu=use_gpu)
return Xp, T

# ----- Minimal self-checks -----
if __name__ == "__main__":
    # Shapes
    L, N = 1000, 20 # loci, samples
    rng = np.random.default_rng(123)
    # Fake centroids: Healthy tighter and more symmetric; Cancer slightly shifted/sk
    alpha_H = rng.uniform(5.0, 15.0, size=L)
    beta_H = rng.uniform(5.0, 15.0, size=L)
    alpha_C = alpha_H + rng.normal(0.5, 0.6, size=L)
    beta_C = beta_H + rng.normal(-0.3, 0.6, size=L)
    alpha_C = np.clip(alpha_C, 0.5, None)
    beta_C = np.clip(beta_C, 0.5, None)
    # Samples as beta means with small noise (you will have real  $\beta$  in  $[0,1]$ )
    mu_H = beta_mean(alpha_H, beta_H)
    mu_C = beta_mean(alpha_C, beta_C)
    X_healthy = np.clip(mu_H[:, None] + rng.normal(0, 0.03, size=(L, N//2)), 0, 1)
    X_cancer = np.clip(mu_C[:, None] + rng.normal(0, 0.05, size=(L, N//2)), 0, 1)
    X = np.concatenate([X_healthy, X_cancer], axis=1) # [L, N]
    # Compute transform and apply
    Xp, T = eat_transform_from_betas(
        X, alpha_H, beta_H, alpha_C, beta_C,
        gamma=1.0, norm="l2", clip_T=10.0,
        use_gpu=False # Set to True if CuPy available
    )
    # Basic sanity checks
    assert T.shape == (L,)
    assert Xp.shape == X.shape
    assert np.isfinite(T).all()
    assert np.isfinite(Xp).all()
    # Quick diagnostic: transformed variance should differ from raw
    raw_var = X.var(axis=0).mean()
    new_var = Xp.var(axis=0).mean()

```

```
print(f"[EAT] mean per-sample variance: raw={raw_var:.6f} transformed={new_var:.6f}")
```